

Alanya Admin — Fonctionnalité Broadcast / Diffusion

Plan technique

15 juillet 2026

1 Objectif

Permettre à un administrateur Alanya d'envoyer des messages (texte, image, vidéo) à tous les utilisateurs ou à un sous-ensemble ciblé. Les messages sont envoyés depuis un compte système « Alanya » avec badge certifié, et affichés avec une UI spéciale dans l'app mobile.

2 Architecture

2.1 Compte Alanya (système)

Créer un user avec un numéro réservé :

- nom = « Alanya »
- pseudo = « alanya »
- alanyaPhone = numéro réservé (ex : 0000)
- type_compte = 0 (apparaît comme user normal côté mobile)
- avatar_url = logo Alanya

Ce compte est le `senderID` de tous les broadcasts.

2.2 Tables SQL

2.2.1 Table broadcast

```
1 CREATE TABLE broadcast (
2   id                INT AUTO_INCREMENT PRIMARY KEY,
3   sender_id         INT NOT NULL,        -- FK users (compte Alanya)
4   created_by        INT NOT NULL,        -- FK users (admin qui diffuse)
5   content            TEXT,
6   type              TINYINT DEFAULT 0,  -- 0=text, 1=image, 2=video
7   media_url          VARCHAR(255),
8   target_type        VARCHAR(20) NOT NULL, -- 'all', 'country', 'specific'
9   target_criteria     JSON,              -- {"idPays":5} ou {"userIds":[1,2,3]}
10  recipient_count    INT DEFAULT 0,
11  sent_at            DATETIME NOT NULL,
12  FOREIGN KEY (sender_id) REFERENCES users(alanyaID),
13  FOREIGN KEY (created_by) REFERENCES users(alanyaID)
14 );
```

2.2.2 Table broadcast_recipient

```
1 CREATE TABLE broadcast_recipient (
2   id                INT AUTO_INCREMENT PRIMARY KEY,
3   broadcast_id       INT NOT NULL,
4   alanya_id          INT NOT NULL,
5   conversation_id    BIGINT,
6   message_id         BIGINT,
7   sent_at            DATETIME NOT NULL,
8   FOREIGN KEY (broadcast_id) REFERENCES broadcast(id),
9   FOREIGN KEY (alanya_id) REFERENCES users(alanyaID),
10  FOREIGN KEY (conversation_id) REFERENCES conversation(conversID),
11  FOREIGN KEY (message_id) REFERENCES message(msgID),
12  UNIQUE (broadcast_id, alanya_id)
13 );
```

2.3 Migration

```
1 migrations/018_broadcast_tables.sql
```

3 Endpoints API

3.1 POST /api/admin/broadcasts — Créer et envoyer

Middleware : adminAuth

Request body :

```
1 {
2   "content": "Message de test",
3   "type": 0,
4   "mediaUrl": null,
5   "targetType": "all",
6   "targetCriteria": {}
7 }
```

Logique :

1. Vérifier admin auth
2. Résoudre les destinataires selon `target_type` :
 - `all` : tous les users non-bannis
 - `country` : users par pays (`targetCriteria.idPays`)
 - `specific` : users spécifiques (`targetCriteria.userIds`)
3. Pour chaque destinataire :
 - Créer ou réutiliser la conversation 1-1 entre « Alanya » et le user
 - Insérer le message dans `message`
 - Mettre à jour `conversation.lastMessage`
 - Incrémenter `conv_participants.unreadCount`
 - Envoyer FCM push via `notifyNewMessage()`
4. Enregistrer dans `broadcast + broadcast_recipient`
5. Retourner le résumé

3.2 GET /api/admin/broadcasts — Historique

Query params : page, limit

Response :

```
1 {
2   "items": [...],
3   "total": 42,
4   "page": 1,
5   "limit": 20
6 }
```

3.3 GET /api/admin/broadcasts/ :id — Détails

Retourne le broadcast + liste des destinataires avec statut d'envoi.

4 Logique de conversation

```
1 // Trouver ou créer la conversation 1-1
2 let conversation = await pool.execute(
3   'SELECT c.conversID FROM conversation c
4   JOIN conv_participants p1 ON p1.conversID = c.conversID
5   JOIN conv_participants p2 ON p2.conversID = c.conversID
6   WHERE c.isGroup = 0
7   AND p1.alanyaID = ? AND p2.alanyaID = ?
8   LIMIT 1',
9   [alanyaSenderId, recipientId]
10 );
```

```

11
12 if (!conversation.length) {
13   // Cr er la conversation
14   const [conv] = await pool.execute(
15     'INSERT INTO conversation (isGroup, lastMessage, lastMessageAt, lastMessageSenderId,
16       lastMessageType)
17       VALUES (0, ?, NOW(), ?, ?)',
18     [preview, alanyaSenderId, type]
19   );
20   conversationId = conv.insertId;
21
22   // Ajouter les participants
23   await pool.execute(
24     'INSERT INTO conv_participants (conversID, alanyaID) VALUES (?, ?), (?, ?)',
25     [conversationId, alanyaSenderId, conversationId, recipientId]
26   );
27 } else {
28   conversationId = conversation[0].conversID;
29   // Mettre jour lastMessage
30   await pool.execute(
31     'UPDATE conversation SET lastMessage=?, lastMessageAt=NOW(), lastMessageSenderId=?,
32       lastMessageType=?
33       WHERE conversID=?',
34     [preview, alanyaSenderId, type, conversationId]
35   );
36 }
37
38 // Envoyer le message
39 const [msg] = await pool.execute(
40   'INSERT INTO message (senderID, conversationID, content, type, status, sendAt, mediaUrl)
41     VALUES (?, ?, ?, ?, 1, NOW(), ?)',
42   [alanyaSenderId, conversationId, content, type, mediaUrl]
43 );
44
45 // Incr menter unreadCount
46 await pool.execute(
47   'UPDATE conv_participants SET unreadCount = unreadCount + 1
48     WHERE conversID = ? AND alanyaID != ?',
49   [conversationId, alanyaSenderId]
50 );

```

5 FCM Push

Utiliser `notifyNewMessage()` existant :

- Type : message
- Title : « Alanya »
- Body : preview du message
- ConversationId : l'ID de la conversation
- CallerId : l'ID du compte Alanya

6 Mobile — UI spéciale

Côté app Talky :

- Détecter que le sender est le compte Alanya (`alanyaPhone` réservé)
- Afficher badge certifié (icône vérifiée)
- Photo de profil = logo Alanya
- Page profil : juste « Alanya » + icône certifiée, rien d'autre
- UI spéciale pour le message (fond différent, badge visible)

7 Frontend Admin — Composants

7.1 Page /broadcasts

- Historique des broadcasts envoyés

- Bouton « Nouveau broadcast »
- Colonnes : date, contenu (truncqué), cible, destinataires, statut

7.2 Dialog Nouveau broadcast

- **Contenu** : textarea + upload image/vidéo
- **Ciblage** :
 - Tous les utilisateurs
 - Par pays (select dropdown)
 - Utilisateurs spécifiques (recherche + sélection multi)
- **Aperçu** : mini preview du message
- **Envoi** : bouton « Diffuser maintenant »

7.3 Types TypeScript

```

1 interface Broadcast {
2   id: number;
3   senderId: number;
4   createdBy: number;
5   content: string;
6   type: number;          // 0=text, 1=image, 2=video
7   mediaUrl: string | null;
8   targetType: string;    // 'all' | 'country' | 'specific'
9   targetCriteria: Record<string, unknown>;
10  recipientCount: number;
11  sentAt: string;
12 }
13
14 interface BroadcastRecipient {
15   id: number;
16   broadcastId: number;
17   alanyaId: number;
18   conversationId: number | null;
19   messageId: number | null;
20   sentAt: string;
21 }
22
23 interface BroadcastFormData {
24   content: string;
25   type: number;
26   mediaUrl?: string;
27   targetType: 'all' | 'country' | 'specific';
28   targetCriteria: {
29     idPays?: number;
30     userIds?: number[];
31   };
32 }

```

8 Hooks React Query

```

1 useBroadcasts(params)           // GET /admin/broadcasts
2 useBroadcastDetail(id)          // GET /admin/broadcasts/:id
3 useCreateBroadcast()             // POST /admin/broadcasts

```

9 Fichiers à créer

1. migrations/018_broadcast_tables.sql
2. src/controllers/admin/broadcast.js
3. src/routes/admin.js (ajouter routes)
4. app/(dashboard)/broadcasts/page.tsx
5. components/broadcasts/BroadcastDialog.tsx
6. components/broadcasts/BroadcastHistory.tsx

7. `components/broadcasts/BroadcastPreview.tsx`
8. `hooks/useBroadcasts.ts`
9. `lib/mock-data.ts` (ajouter fonctions)
10. `types/index.ts` (ajouter types)
11. `app/(dashboard)/layout.tsx` (ajouter nav)